

Лабораторная работа № 7.

Введение в фильтрацию изображений

Цель работы – освоение навыков фильтрации растровых изображений с помощью алгоритмических языков программирования.

Краткие теоретические сведения

В процессе формирования изображения оптический сигнал подвергается искажениям, которые проявляются тем, что точечный источник сцены при проецировании на поверхность фотоприёмника точкой выглядеть уже не будет, а превратится в размытое пятно. Функция яркости этого пятна $h(x', y')$ называется функцией рассеяния точки (ФРТ). Если представить сцену как совокупность точек различной яркости, то с учётом этих искажений изображение каждой из этих точек на фотоприёмнике будет размытым, а в результате и изображение сцены на фотоприёмнике как совокупности этих точек будет размытым. Характер такого рода искажений зависит от условий получения изображения и характеристик среды распространения оптического сигнала, оптической системы и датчика изображения.

Вследствие свойств оптического излучения и применяемых материалов, за исключением специальных случаев, такие преобразования относятся к линейным, так как они обладают свойствами однородности и аддитивности. Преобразование называется однородным, если изменение сигнала на входе порождает пропорциональное изменение сигнала на выходе. Преобразование называется аддитивным, если преобразование суммы сигналов равно сумме преобразований сигналов. Такие преобразования можно рассматривать как фильтрацию сигнала в рамках теории обработки сигналов. Компоненты оптической системы, обуславливающие эти искажения, могут быть описаны как фильтры. В качестве сигнала рассматривается пространственно-зависимая функция яркости $f(x, y)$. Результат линейного преобразования будет выражаться суперпозицией функции яркости сцены и функции рассеяния точки:

$$g(x', y') = \iint_{-\infty}^{\infty} f(x, y) h(x, y, x', y') dx dy,$$

где x', y' – координаты на плоскости фотоприёмника; $g(x', y')$ – функция яркости на плоскости фотоприёмника; x, y – координаты сцены,

$f(x, y)$ – функция яркости сцены; $h(x, y, x', y')$ – функция рассеяния точки, форма которой описывается как функция (x', y') и в общем случае может зависеть от положения (x, y) на сцене, другими словами, $h(x', y')$ может быть разной для точек в центре сцены и на периферии сцены, соответственно, зависит также и от значений (x, y) . В большинстве случаев принято допускать, что это не так, преобразование является пространственно-инвариантным, функция рассеяния точки везде одинаковая и выражается как функция двух переменных, — зависимость $h(x'-x, y'-y)$ от разности координат на сцене и на изображении, а интеграл суперпозиции превращается в интеграл свёртки:

$$g(x', y') = (f * h)(x', y') = \iint_{-\infty}^{\infty} f(x, y) h(x - x', y - y') dx dy.$$

Свёртка обладает свойствами коммутативности, ассоциативности, дистрибутивности относительно сложения и умножения на константу. Дифференциал свёртки двух функций равен свёртке одной функции с дифференциалом другой.

Операция свёртки связана с преобразованием Фурье: образ Фурье результата свёртки равен произведению образов Фурье свёртываемых функций: $F\{f * h\} = F\{f\} \cdot F\{h\}$. Симметрично образ Фурье произведения функций равен свёртке образов Фурье функций: $F\{f \cdot h\} = F\{f\} * F\{h\}$.

Преобразование Фурье – это преобразование функции $f(x)$ вида

$$F(u) = F\{f(x)\} = \int_{-\infty}^{\infty} f(x) e^{-2\pi j u x} dx$$

при соблюдении условий существования интеграла. В результате преобразования получается образ Фурье $F(u)$ функции $f(x)$. Предполагается, что $f(x)$ и её образ — комплексные функции. Если $f(x)$ – действительная, то мнимая часть её значений будет везде равна нулю, но её образ Фурье будет действительным только тогда, когда $f(x)$ чётная.

Экспонента в формуле преобразования Фурье может быть выражена в виде $e^{-2\pi j u x} = \cos(2\pi u x) + j \sin(2\pi u x)$. Частота синусоиды задаётся u , а в процессе преобразования для каждого значения u измеряется интегральный вес синусоид этой частоты в сигнале $f(x)$. Синусоида каждой фиксированной частоты u называется гармоникой. Образ Фурье представляет собой функцию $F(u)$, которая показывает вес гармоники с частотой u в сигнале $f(x)$. Можно сказать, что Фурье образ представляет сигнал в виде совокупности синусоид различной частоты, каждая со своим весом. Эта совокупность называется также

спектром сигнала. Спектр содержит всю информацию о сигнале, и сигнал может быть выражен из спектра путём обратного преобразования Фурье: $f(x) = F^{-1}\{F(u)\} = \int_{-\infty}^{\infty} F(u)e^{2\pi jux} du$ с точностью до постоянного множителя.

Двумерное преобразование Фурье $F(u, v) = \iint_{-\infty}^{\infty} f(x, y)e^{-2\pi jux-2\pi jvy} dx dy$ обладает теми же свойствами, что и одномерное. Аналогично выполняется преобразование Фурье для любого количества измерений.

Вес гармоник зависит от скорости изменения сигнала. Медленные изменения сигнала соответствуют гармоникам низкой частоты, быстрые – высокой. На изображении резкий перепад яркости описывается быстрым изменением функции яркости, что добавляет веса гармоникам высокой частоты. Обычно изменения оптического сигнала такого рода возникают на контрастных границах объектов. Многие шумы на изображении также проявляются резкими скачками сигнала. Для представления плавных изменений функции яркости достаточно гармоник из ограниченного диапазона низких частот – от 0 до частоты среза $u_{гр}$.

В процессе фильтрации часть гармоник пропускается, другая часть — задерживается. Фильтры, пропускающие гармоники, частота которых находится в пределах от 0 до $u_{гр}$, т. е. только низкие частоты, называются фильтрами низких частот (ФНЧ). Фильтры, пропускающие только высокие частоты от $u_{гр}$ до бесконечности, называются фильтрами высоких частот (ФВЧ). При этом, рассматривая образ Фурье сигнала, вес тех гармоник, которые должны быть пропущены, умножается на 1, а вес тех, которые нужно убрать из сигнала, — на 0. При умножении на величину от 0 до 1 гармоника ослабляется, от 1 до бесконечности — усиливается. Функция $H(u)$, определяющая, какие компоненты сигнала и в какой степени должны пропускаться фильтром, называется частотной функцией. Спектр отфильтрованного сигнала вычисляется как $F(u) \cdot H(u)$. Обратным преобразованием Фурье получается сам сигнал.

Строго говоря, фильтр характеризуется передаточной функцией $H(s)=B(s)/A(s)$, где $A(s)$ и $B(s)$ — результаты преобразования Лапласа $F(s) = \int_{-\infty}^{\infty} f(x)e^{-sx} dx$ входного и выходного сигналов соответственно, или $H(z)=B(z)/A(z)$, где $A(z)$ и $B(z)$ — результаты z -

преобразования входного и выходного сигналов соответственно – частного случая преобразования Лапласа. Преобразование Фурье тоже является частным случаем преобразования Лапласа, полученное путём замены s на ju , так, чтобы речь шла о частоте гармоник. Частотная функция фильтра $H(u)$ представляет собой частный случай передаточной функции.

Применение ФНЧ ликвидирует из сигнала резкие переходы, что приводит к размыванию границ, так как в сигнале остаётся информация только о плавных изменениях. В то же время, если шум выражается только резкими скачками величины сигнала, он будет удалён. Применение ФВЧ оставит в сигнале только резкие переходы, в результате чего останется информация только о границах. Шум в виде резких скачков также будет пропущен. Кроме ФНЧ и ФВЧ известны полосовые фильтры, пропускающие сигнал, содержащий частоты из определённого диапазона, и режекторные фильтры, которые пропускают все частоты, кроме частот из определённого диапазона.

Произведение образа Фурье сигнала на передаточную функцию $H(u)$ соответствует свёртке самого сигнала с функцией $h(x)$, которую можно получить обратным преобразованием Фурье передаточной функции. Функция h называется импульсной характеристикой фильтра. Для изображения ФРТ является импульсной характеристикой, реализующей ФНЧ. Описание работы фильтра в виде произведения образа Фурье сигнала на передаточную функцию и в виде свёртки сигнала с импульсной характеристикой эквивалентны. При выполнении программной фильтрации изображений применяется способ, наиболее удобный в данных условиях. В ряде случаев за счёт эффективных алгоритмов дискретного быстрого преобразования Фурье (БПФ) оказывается быстрее найти образы Фурье сигнала и фильтра, перемножить их и выполнить обратное преобразование Фурье для получения результата. В других случаях проще и быстрее использовать свёртку, для реализации которой также известны быстрые способы вычислений.

Дискретная двумерная свёртка выполняется по формуле $g_{k,l} = \sum_i \sum_j f_{i,j} h_{k-i,l-j}$. Цифровое изображение размера $M \times N$ представлено в виде дискретной функции яркости $f_{i,j} = f(i,j)$, $i = 0, \dots, M-1$, $j = 0, \dots, N-1$. Двумерная импульсная характеристика (ядро) фильтра в дискретном виде представлена как

матрица $h_{k,l}=h(k,l)$ размера $P \times Q$. С учётом конечности массивов изображения и ядра, свойства коммутативности свёртки, положив, что центральный элемент матрицы свёртки (точка привязки) имеет координаты $(0,0)$, формулу можно преобразовать в более удобный для вычислений вид: $g_{i,j} = \sum_{k=-P/2}^{P/2} \sum_{l=-Q/2}^{Q/2} f_{i-k,j-l} h_{k,l}$.

Для однозначного определения позиции центрального элемента удобнее, когда ядро имеет нечётные размеры. В случае необходимости ядро можно дополнить нулями. Обычно ядро задаётся в виде матрицы размером от 2×1 до 9×9 , иногда и более. Размер и форма ядра (апертура) определяют количество пикселей входного изображения, участвующих в получении каждого пикселя выходного изображения. Увеличение размеров матрицы позволяет более точно определить характеристики фильтра, но приводит к увеличению объёма вычислений и усилению краевых эффектов, когда области, близкие к краю изображения, не могут быть полноценно обработаны.

Краевые эффекты возникают вследствие того, что при $i < P/2$, $j < Q/2$, $i > M-P/2$, $j > N-Q/2$ точек исходного изображения $f_{i-k,j-l}$ не существует, так что для выходного изображения $i=P/2, \dots, M-P/2-1$, $j=Q/2, \dots, N-Q/2-1$, то есть в результате свёртки изображение уменьшается на размер ядра по каждому измерению. Краевые эффекты можно снизить без уменьшения изображения, если, например, тем или иным манером добавить по краям изображения кайму из крайних пикселей изображения шириной в половину размера фильтра, но при этом качество фильтрации в этой области всё равно ухудшится из-за фактической нехватки информации. Краевые эффекты связаны с тем, что при выполнении преобразования Фурье либо оригинал, либо Фурье образ может быть финитной функцией, но не одновременно. При этом фактически как спектр оцифрованного сигнала ограничен, так и сам сигнал ограничен, так что, либо в сигнале, либо в спектре часть информации не представлена.

При реализации фильтра в виде свёртки, для получения значения каждого пикселя $g_{i,j}$ выходного сигнала, ядро фильтра накладывается центром на соответствующий пиксель $f_{i,j}$ входного сигнала. Значения пикселей входного сигнала, покрытые апертурой фильтра, перемножаются на соответствующие элементы матрицы ядра. Полученные произведения суммируются, и результат записывается в выходной сигнал.

Часто процесс свёртки рассматривается как движение окна, совпадающего с апертурой фильтра, которое выбирает на исходном изображении группу пикселей, участвующих в свёртке. Окно передвигается на один пиксель по горизонтали или вертикали. Пиксели, попавшие в окно, свёртываются с ядром фильтра и формируют один элемент выходного изображения.

Если ядро фильтра можно представить в виде произведения столбца на строку $h(k, l) = h_1(k) \cdot h_2(l)$, такой фильтр называется сепарабельным. Обработку таким фильтром можно производить отдельно: сначала столбцом обработать каждый столбец изображения, затем каждую строку результата обработать строкой фильтра.

К линейным преобразованиям относятся интегрирование (в том числе и с весом), дифференцирование, умножение функций, вычисление функции в одной точке сигнала и с некоторыми оговорками, аффинные преобразования. Линейные пространственно инвариантные преобразования могут быть описаны как фильтры и реализованы в виде свёртки. Например, дифференцирование представляет собой фильтр высоких частот.

Если преобразование не обладает свойством однородности или аддитивности, оно относится к нелинейным. Теоретически нелинейные преобразования исследованы в меньшей мере, чем линейные, что ограничивает разработку и применение соответствующих фильтров. Нелинейные преобразования невозможно описать такими формализованными средствами, как свёртка или суперпозиция. Нелинейная фильтрация выполняется по специальному алгоритму, который может быть достаточно простым.

Одним распространённых нелинейных фильтров является медианный. Для медианной фильтрации, как и при свёртке с матрицей, рассматривается окно вокруг пикселя. В качестве результата берётся медианное значение сигнала в окне. Частотная характеристика медианного фильтра зависит от входных данных: если там присутствуют единичные выбросы, они будут удалены, т. е. частотная характеристика обеспечит существенное снижение высоких частот; более однородные исходные данные будут фильтроваться более мягко.

Фильтры с конечной импульсной характеристикой (КИХ) реализуются в виде свёртки входного сигнала с ядром конечного размера (рис. 1). Так как финитной функцией может быть либо оригинал, либо

образ Фурье, у КИХ-фильтра частотная характеристика не будет нулевой на интервале от минус бесконечности до бесконечности. Следовательно, никакие частоты сигнала таким фильтром не будут подавляться полностью.

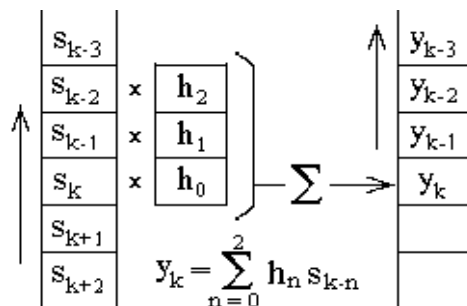


Рис. 1. Схема работы КИХ-фильтра

Если необходимо пропустить частоты из определённого диапазона и полностью задержать все остальные частоты сигнала, требуется фильтр, частотная характеристика которого будет принимать за пределами области пропускания строго нулевые значения, т. е. будет финитной. Тогда импульсная характеристика должна быть бесконечной. Фильтры с бесконечной импульсной характеристикой (БИХ) реализуются путём свёртки за счёт введения обратной связи (рис. 2).

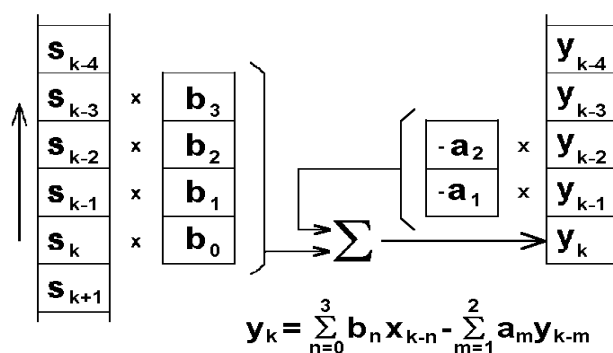


Рис. 2. Схема работы БИХ-фильтра

Такие фильтры лучше подавляют частоты, которые не должны пропускаться. Их труднее рассчитать и реализовать.

Для обработки изображений в большинстве случаев используются КИХ-фильтры. Это связано также и с тем, что по природе изоб-

ражения пиксели могут быть обработаны в любой последовательности. При использовании БИХ-фильтра результат будет зависеть от направления обработки.

Результат свёртки принимает максимальное значение в тех позициях, где форма сигнала совпадает с формой функции, описывающей ядро свёртки. Если ядро свёртки задать в соответствии с функцией яркости объекта, в результате свёртки в позиции объекта будет наблюдаться локальный максимум. Частотная характеристика такого фильтра будет равна произведению выражения, комплексно сопряжённого с выражением, описывающим спектр объекта на $e^{-j(ua+vb)}$, $a \times b$ – размер объекта.

Для определения свёрточного фильтра следует задать матрицу чисел определённого размера. Чаще всего используется размер 3×3 . Эти числа должны представлять функцию, являющуюся обратным преобразованием Фурье частотной функции фильтра.

Матрица может состоять из действительных чисел любой величины, и это может привести к тому, что у результата значения выйдут за пределы допустимого. В большинстве случаев яркость элемента находится в пределах от 0 до 255. При свёртке с ядром, в состав которого входят значения большой величины, результат может превысить 255, а при свёртке с ядром, содержащим отрицательные значения, результат тоже может оказаться меньше нуля. Известны по меньшей мере два способа решения этой проблемы.

1. Преобразовать ядро свёртки так, чтобы при сохранении формы импульсной характеристики все элементы были в пределах от 0 до 1, а сумма всех элементов ядра была равна единице.

2. Преобразовать результат нужным способом. В простейшем случае, если ядро не содержит отрицательных элементов, — разделить каждое выходное значение на сумму элементов ядра.

Один из простейших фильтров — усредняющий. По сути он применяется путём вычисления среднего значения некоторой окрестности точки или, можно сказать, среднего значения пикселей, попавших в окно и представляет собой определённый интеграл сигнала по области окна. Это преобразование можно определить как свёртку с ядром, состоящим из одних единиц с последующим делением на количество элементов ядра. Частотная характеристика такого фильтра будет иметь вид

$H(u, v) = \frac{\sin u}{u} \frac{\sin v}{v}$, т. е. это будет плохой фильтр низких частот, пропускающий, хоть и в меньшей мере, чем низкие, все частоты до бесконечности.

Хороший ФНЧ будет пропускать только низкие частоты до частоты среза u_{cp} , а остальные — задерживать. Частотная характеристика у него будет прямоугольной, а импульсная, соответственно, будет иметь вид $h(x, y) = \frac{\sin x}{x} \frac{\sin y}{y}$. Поскольку эта функция убывает медленно, соответствующее ядро преобразования должно быть большим, следовательно, путём свёртки такой фильтр реализовывать затратно, а краевые эффекты будут велики по-любому.

Удачной формой частотной характеристики оказался гауссиан $H(u) = e^{-u^2}$. Гауссиан является чётной функцией. В результате преобразования Фурье получается тоже гауссиан. Хотя эта функция нефинитна, она быстро убывает, что позволяет эффективно ограничить частоту при небольших размерах ядра свёртки. К достоинствам этой функции относится и то, что искажения оптической системы типа расфокусировки и аберраций, приводящие к снижению чёткости изображения, хорошо приближаются функцией именно такого вида. Двумерное ядро свёртки в форме гауссиана описывается формулой $h(x, y) = \frac{1}{2\pi\sigma^2} e^{\frac{-x^2+y^2}{2\sigma^2}}$. У гауссова фильтра всего один параметр σ , определяющий ширину полосы пропускания: чем он больше, тем уже полоса, тем больше степень расфокусировки. Матрицу для свёртки можно получить, вычислив её, например, с помощью табличного процессора или заполнив с помощью формулы массив в программе. Количество элементов матрицы обычно не берут больше, чем 64. Часто вещественные числа округляют до целых, предварительно взвешивая их удобными коэффициентами. На практике, если матрица вычисляется прямо в программе по формуле, в этом нет необходимости. Пример вычисления гауссовой матрицы свёртки — на рисунке 3, σ — в ячейке A1.

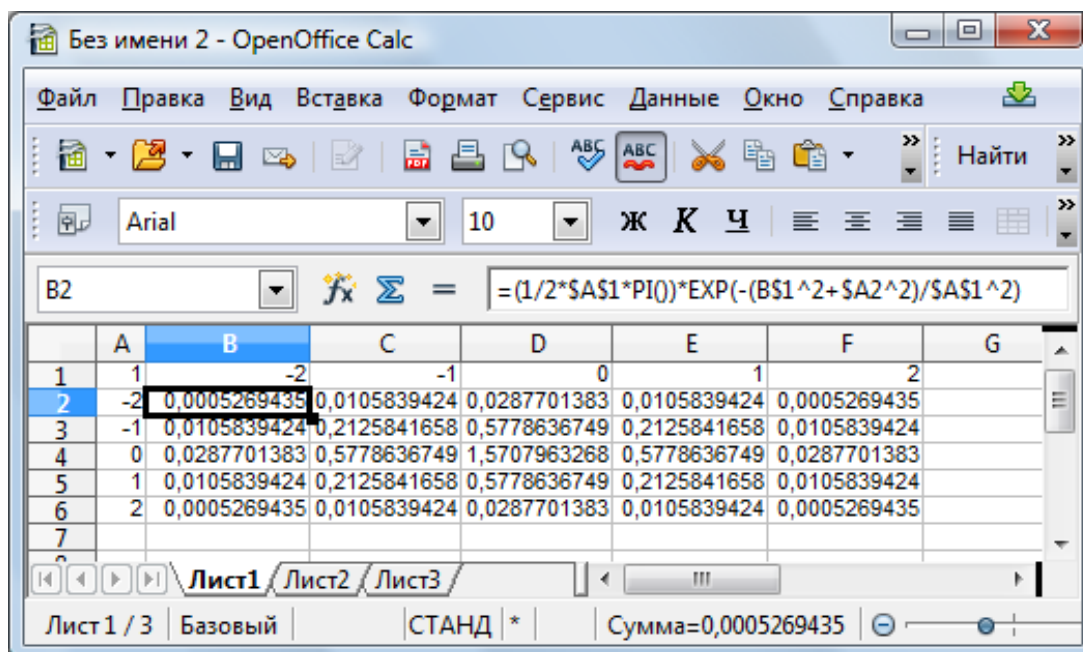


Рис. 3. Вычисление матрицы свёртки в табличном процессоре

Границы объектов на изображении часто характеризуются скачками функции яркости и представляют значительный интерес. Скорость изменения значения функции яркости показывает производная функции яркости. Численной оценкой частной производной двумерной функции по направлению слева направо может служить конечная разность $\frac{\partial f(x,y)}{\partial x} \approx f_{i+1,j} - f_{i,j}$ или симметричная конечная разность $\frac{\partial f(x,y)}{\partial x} \approx f_{i+1,j} - f_{i-1,j}$. Дискретное ядро свёртки для реализации конечной разности может иметь размер 2×1 , а для реализации центральной конечной разности — 3×1 . В отличие от ранее рассмотренных фильтров ядро дифференцирующего фильтра асимметрично и применение фильтра в разных направлениях будет приводить к разным результатам. Кроме того, невозможно определить производную в точке сразу по всем направлениям. Для оценки производной по другим направлениям соответствующим образом выбираются координаты точек. Для дискретизованного на регулярной прямоугольной решётке изображения можно определить четыре направления: слева направо, сверху вниз и две диагонали. Производные в этих направлениях можно оценить свёрткой с соответствующими ядрами размером 2×1 и 2×2 . Свёртка, реализующая центральную конечную разность, позволяет оценить производные по восьми направлениям. Есть и другие способы оценки производных.

Дифференцирующий фильтр хорошо будет пропускать границы, но ещё лучше — помехи, поэтому перед применением фильтра высоких частот изображение должно быть как можно лучше очищено от шума. Результат свёртки может быть как положительным, так и отрицательным в зависимости от того, возрастает или убывает функция яркости в заданном направлении, так что после его применения результат следует вогнать в пределы допустимого.

Логично предположить, что граница находится там, где скорость изменения функции яркости максимальна. Производная в этой позиции принимает максимальное или минимальное значение. Вторая производная в этих точках равна нулю. Численной оценкой второй частной производной по одной из переменных двумерной функции яркости может служить разность соседних значений первой производной $\frac{\partial^2 f(x,y)}{\partial x^2} \approx -f_{i+1,j} + 2f_{i,j} - f_{i-1,j}$. Ядро свёртки для такой оценки будет симметричным, а фильтр — изотропным. Сумма вторых частных производных двумерной функции является лапласианом функции яркости и принимает нулевое значение, когда первые производные достигают экстремума одновременно по направлениям слева направо и сверху вниз. Оценка лапласиана $\frac{\partial^2 f(x,y)}{\partial x^2} + \frac{\partial^2 f(x,y)}{\partial y^2} \approx 4f_{i,j} - f_{i+1,j} - f_{i-1,j} - f_{i,j+1} - f_{i,j-1}$ содержит пять слагаемых. Оценка суммы вторых производных по четырём направлениям будет содержать девять слагаемых. Все эти оценки легко вычисляются свёрткой с соответствующим ядром.

Частотная характеристика лапласиана имеет ещё более крутой подъём в области высоких частот, а это значит, что реакция лапласиана на помехи будет ещё более выраженной, а предварительная фильтрация от помех — ещё более актуальной. В какой-то мере решить проблему можно, предварительно обработав сигнал фильтром низких частот с помощью, например, гауссиана. Благодаря свойствам ассоциативности и коммутативности свёртки нет разницы, обрабатывать ли сигнал последовательной свёрткой с двумя ядрами или свернуть сигнал один раз с предварительно вычисленным результатом свёртки двух ядер друг с другом. Другими словами, вместо того, чтобы обработать изображение сначала ядром гауссиана, а потом ядром лапласиана, можно предварительно вычислить ядро лапласиана гауссиана, а потом

свернуть с ним изображение. Такое ядро будет одновременно отфильтровывать изображение от шума как гауссиан и определять положение границ как лапласиан.

Дифференцирующий фильтр, лапласиан и лапласиан гауссиана являются фильтрами высоких частот. Результатом их применения будут изображения, содержащие только области границ и лишённые подробностей в однородных областях из-за того, что в результате фильтрации спектр изображения остаётся без низких частот. Для того чтобы просто повысить резкость изображения, необходимо в частотной области усилить высокие частоты, сохранив при этом низкие. Этого можно добиться, сложив исходное изображение с результатом обработки фильтром высоких частот. Фильтрацию и сложение можно совместить, так что в результате применения этой пары операций каждый элемент на выходе будет вычисляться как $f(x, y) + \frac{\partial^2 f(x, y)}{\partial x^2} + \frac{\partial^2 f(x, y)}{\partial y^2} \approx 5f_{i,j} - f_{i+1,j} - f_{i-1,j} - f_{i,j+1} - f_{i,j-1}$ и породить соответствующую маску.

Линейные ФНЧ применяются:

1. Для очистки от помех, если спектр изображения в основном ниже частоты среза фильтра, а спектр шума — в основном выше.
2. Для сглаживания изображения перед его масштабированием, в частности, при построении пирамиды изображений для кратно-масштабного анализа.
3. С целью обеспечения выполнения условий теоремы Котельникова перед передискретизацией.
4. Для коррекции неравномерной освещённости или виньетирования, так как спектр связанного с этим изменения яркости лежит в области очень низких частот.
5. Для создания художественных эффектов.

Линейные ФВЧ применяются:

1. Для поиска границ объектов: детекторы границ Прюитт, Собела, Робертса, Лапласа, Марра, Щарра, Кэнни, Айверсона, Ротуэлла базируются на оценке производных, в т. ч. для вычисления градиентов, в большинстве случаев в комбинации с ФНЧ.
2. Для восстановления изображения, искажённого расфокусировкой и другими аберрациями, снижающими долю высоких частот.
3. Для создания художественных эффектов.

Режекторные фильтры применяются для подавления шума, когда сигнал помехи лежит в узком диапазоне частот, который перекрывается

полезным сигналом. Полосовые фильтры применяются, когда полезный сигнал лежит в узком диапазоне частот.

Матрицы свёртки некоторых фильтров приведены в таблице 1.

Нелинейные фильтры применяются в основном для очистки от помех. Кроме того, некоторые способы обработки, которые могут интерпретироваться как нелинейные, могут быть получены из линейных. Например, градиент, который можно оценить с помощью производных.

Согласованные фильтры применяются для поиска объектов. Это имеет смысл, если объект на изображениях не меняет размер и ориентацию и затруднено, когда размер объекта велик.

Таблица 1

Распространённые матрицы свёртки линейных фильтров

Название	Ядро											
1. Усредняющий	1				1				1			
	1				1				1			
	1				1				1			
2. Прямоугольный	0	0	0	0	1	1	1	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	-1	-2	-1	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0
	1	0	-1	0	4	6	4	0	-1	0	1	1
	1	0	-2	0	6	10	6	0	-2	0	1	1
	1	0	-1	0	4	6	4	0	-1	0	1	1
	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	-1	-2	-1	0	0	0	0	0
	0	0	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	1	1	1	0	0	0	0	0
3. Гауссов	0			1			2			1		
	1			6			10			6		
	2			10			16			10		
	1			6			10			6		
	0			1			2			1		

Окончание табл. 1

Название	Ядро						
4. Дифференцирующий Робертса	по x		по y	по xy		по yx	
	1	-1	1	1	0	0	-1
			-1	0	-1	1	0
5. Дифференцирующий Прюитт	Вертикальный			Горизонтальный			
	-1	-1	-1	-1	0	1	
	0	0	0	-1	0	1	
	1	1	1	-1	0	1	
6. Дифференцирующий Собела	Вертикальный			Горизонтальный			
	-1	-2	-1	-1	0	1	
	0	0	0	-2	0	2	
	1	2	1	-1	0	1	
7. Лапласиан	2 напр.			4 напр.			
	0	-1	0	-1	-1	-1	
	-1	4	-1	-1	8	-1	
	0	-1	0	-1	-1	-1	
8. Лапласиан гауссиана	-1	-2	-3	-2	-1		
	-2	0	4	0	-2		
	-3	4	16	4	-3		
	-2	0	4	0	-2		
	-1	-2	-3	-2	-1		
9. Повышение резкости	2 напр.			4 напр.			
	0	-1	0	-1	-1	-1	
	-1	5	-1	-1	9	-1	
	0	-1	0	-1	-1	-1	
10. Тиснение	0		1		0		
	1		0		-1		
	0		-1		0		

Задание

1. Разработать алгоритм и программу для реализации индивидуального задания. Программа должна включать в себя:
 - обработку изображения свёрточным фильтром низких частот;
 - обработку изображения свёрточным фильтром высоких частот.
2. Ответить на контрольные вопросы.
3. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Формулировка задачи.
3. Алгоритм.
4. Программа на алгоритмическом языке программирования.
5. Исходное и преобразованные изображения.
6. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта, порядком выполнения работы, индивидуальным вариантом, примером выполнения работы, примером оформления отчёта.
2. Ознакомление с теоретическими материалами по теме.
3. Разработка алгоритма в соответствии с вариантом задания. Алгоритм должен предусматривать:
 - чтение файла изображения;
 - обработку фильтром низких частот в соответствии с заданием;
 - обработку фильтром высоких частот в соответствии с заданием;
 - запись преобразованного изображения в файл.
4. Разработка и отладка программы для решения задачи.
5. Ответы на контрольные вопросы.
6. Оформление отчёта.

Индивидуальные задания

Каждый вариант задания (табл. 2) включает в себя применение разных ФВЧ и ФНЧ к разным частям изображения.

Таблица 2

Варианты индивидуальных заданий

№	ФНЧ	ФВЧ
1	К левой половине изображения примените ФНЧ № 1, к правой – ФНЧ № 2	К левой половине изображения примените ФВЧ Прюитт, к правой — ФВЧ Собела.
2	Область изображения за пределами круга радиуса R размойте гауссианом	Повысьте резкость в районе пикселей, превышающих яркость T
3	Размойте пиксели, не превышающие порог яркости T	Повысьте резкость изображения после ФНЧ
4	Уменьшите размеры изображения вдвое, взяв в результат каждый второй пиксель по каждому направлению. Сделайте для сравнения то же самое после обработки исходного изображения ФНЧ	К левой половине изображения примените ФВЧ Робертса, к правой — ФВЧ Прюитт
5	Обработайте изображение двумя гауссианами с разными значениями параметра и найдите разность результатов	К левой половине изображения примените ФВЧ Робертса, к правой — ФВЧ Собела
6	Обработайте изображение ФНЧ № 1 и вычтите результат обработки из исходного изображения	К левой половине изображения примените ФВЧ Лапласа, к правой — лапласиан гауссиана
7	К левой половине изображения примените ФНЧ только для одного из цветовых каналов, к правой — только для другого	К левой половине изображения примените лапласиан гауссиана, к правой — его же, предварительно переведя изображение в оттенки серого
8	К левой половине изображения примените ФНЧ только для яркости, к правой — только для насыщенности	К левой половине изображения примените лапласиан гауссиана только для яркости, к правой — только для насыщенности

Продолжение табл. 2

№	ФНЧ	ФВЧ
9	К левой половине изображения примените ФНЧ только для яркости, к правой — только для цветового тона	К левой половине изображения примените лапласиан гауссиана только для яркости, к правой — только для насыщенности
10	К левой половине изображения примените ФНЧ только для насыщенности, к правой — только для цветового тона	К левой половине изображения примените лапласиан гауссиана только для яркости, к правой — только для насыщенности
11	К левой половине изображения примените ФНЧ № 2, к правой — ФНЧ № 3	К левой половине изображения примените лапласиан гауссиана только для яркости и цветового тона, к правой — только для насыщенности и цветового тона
12	Область изображения за пределами прямоугольника axb размойте гауссианом	К левой половине изображения примените лапласиан гауссиана только для яркости и цветового тона, к правой — только для яркости и насыщенности
13	Размойте пиксели, превышающие порог яркости T	К левой половине изображения примените лапласиан гауссиана только для яркости и насыщенности, к правой — только для насыщенности и цветового тона
14	Увеличьте размеры изображения вдвое, взяв в результат каждый пиксель по каждому направлению по два раза. Обработайте результат ФНЧ	К левой половине изображения примените лапласиан гауссиана, к правой — его же, предварительно переведя изображение в негатив, а после — обратно
15	Обработайте изображение двумя ФНЧ № 1 и ФНЧ № 3, найдите разность результатов	К изображению примените ФВЧ Лапласа и лапласиан гауссиана. Найдите разность результатов

№	ФНЧ	ФВЧ
16	Обработайте изображение ФНЧ № 3 и вычитите результат обработки из исходного изображения	К изображению примените ФВЧ Робертса и ФВЧ Собела. Найдите разность результатов
17	К левой половине изображения примените ФНЧ только для двух из трёх цветовых каналов, к правой — только для других двух.	К изображению примените ФВЧ Робертса и ФВЧ Прюитт. Найдите разность результатов.
18	К левой половине изображения примените ФНЧ только для яркости и цветового тона, к правой — только для насыщенности и цветового тона	Повысьте резкость изображения в пределах круга радиусом R
19	К левой половине изображения примените ФНЧ только для яркости и насыщенности, к правой — только для яркости и цветового тона	Повысьте резкость в районе пикселей, не превышающих яркость T
20	К левой половине изображения примените ФНЧ только для насыщенности и яркости, к правой — только для насыщенности и цветового тона	К изображению примените ФВЧ Прюитт, и ФВЧ Собела. Найдите разность результатов

Контрольные вопросы

1. Какие фильтры проходит оптический сигнал в процессе формирования на поверхности фотоприёмника двумерного изображения трёхмерной сцены и к какому классу относится каждый из этих фильтров?

2. Какие фильтры проходит оптический сигнал в процессе формирования цифрового изображения от поверхности фотоприёмника до памяти и какому классу относится каждый из этих фильтров?
3. Какие из потерь в процессе фильтрации необратимы и почему?
4. Какие из потерь в процессе фильтрации обратимы и как?
5. В чём преимущества и недостатки БИХ-фильтров по сравнению с КИХ в области обработки изображений?
6. Как реализовать БИХ-фильтр с помощью свёртки с окном?
7. Как реализовать попиксельные преобразования с помощью свёртки?
8. Как реализовать геометрические преобразования с помощью свёртки?
9. Какие эффекты фотошопа реализованы путём фильтрации?

Пример выполнения работы

0	Размыть изображение ФНЧ Гаусса с параметром, увеличивающимся по мере удаления от центра	По каждому каналу увеличить резкость точек, для которых контраст под окном превышает заданный порог
---	---	---

При выполнении первой части задания расстояние от обрабатываемой точки до центра будем вычислять по теореме Пифагора. В зависимости от расстояния будет изменяться параметр рассеяния. Чем больше параметр рассеяния, тем больший размер ядра необходим для представления гауссовского фильтра. Если ограничиться размером ядра 15×15 , то параметр рассеяния следует брать около $15/6=2,5$. В крайнем случае до 6, при этом фильтр на большом удалении от центра будет не совсем гауссовским.

При выполнении второй части задания в качестве фильтра для повышения резкости возьмём четырёхнаправленный. Размер ядра 3×3 , и контраст вычислим в пределах девяти точек. Контраст будем вычислять для каждой цветовой компоненты отдельно, как разницу между максимальным и минимальным значением яркости.

На форме (рис. 4) предусматриваем по кнопке и диалогу для считывания изображения и сохранения каждого из двух результатов, три объекта Tgraphics для хранения и отображения картинок, кнопки для двух преобразований, поле ввода для порога контраста.

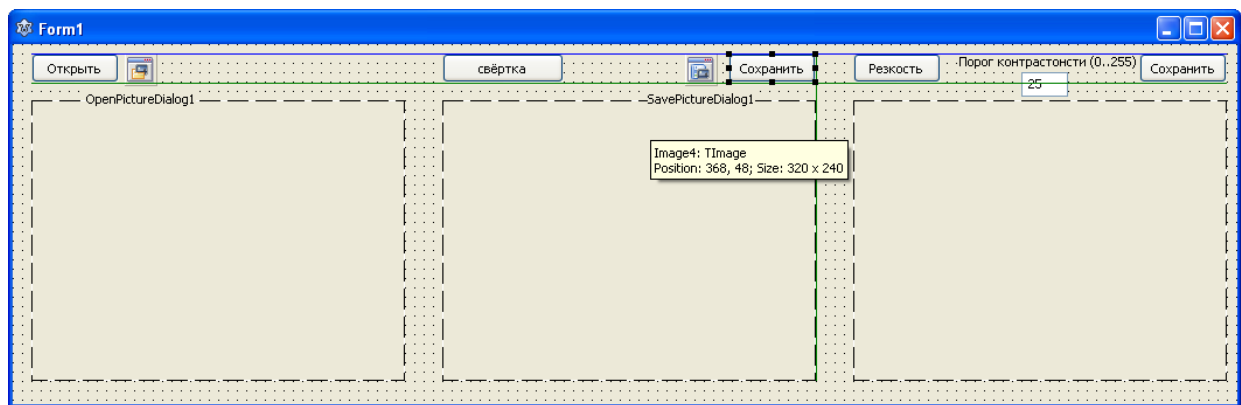


Рис. 4. Форма для программы фильтрации изображений

Последовательность действий для первого преобразования:

1. Считываем изображение, преобразовываем в полноцветное.
2. Инициализируем изображение с теми же параметрами, что и исходное.
3. Вычисляем коэффициент с которым будет увеличиваться параметр рассеяния фильтра при увеличении расстояния от центра: $kp := 6 / \sqrt{(wi/2 - kw/2)^2 + (he/2 - kh/2)^2}$, где wi , he — длина и ширина изображения; kw , kh — размеры ядра свёртки.
4. Для каждого обрабатываемого пикселя вычисляем параметр рассеяния: $s := kp * \sqrt{(i - wi/2 + kw/2)^2 + (j - he/2 + kh/2)^2}$, с этим параметром вычисляем значения ядра свёртки, выполняем свёртку окрестностей пикселя с вычисленным ядром.
5. Проверяем, не вышли ли результаты за пределы допустимых уровней яркости, и корректируем при необходимости.
6. Выводим результат.
7. Предусматриваем возможность записи результатов.

Последовательность действий для второго преобразования:

1. Инициализируем изображение с теми же параметрами, что и исходное.
2. Считываем пороговое значение контраста.
3. Формируем ядро преобразования.
4. Формируем массивы исходных данных и данных результата.
5. Для каждого обрабатываемого пикселя по каждому каналу в окрестностях, соответствующих окну преобразования, находим минимальное и максимальное значение. Если разница между этими значениями превышает заданный порог, осуществляем свёртку с ядром.

6. Проверяем, не вышли ли результаты за пределы допустимых уровней яркости, и корректируем при необходимости.

7. Выводим результат.

8. Предусматриваем возможность записи результатов.

Получаем модуль приблизительно следующего содержания:

```
unit Unit1;
```

```
{ $mode objfpc } { $H+ }
```

```
interface
```

```
uses
```

```
Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs, StdCtrls,  
GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls, Ex-  
tDlgs, Menus,  
Grids, fpImage, math, crt;
```

```
type
```

```
 { TForm1 }
```

```
TA = Array of array of real;
```

```
TForm1 = class(TForm)
```

```
  Button1: TButton;
```

```
  Button2: TButton;
```

```
  Button4: TButton;
```

```
  Button5: TButton;
```

```
  Button6: TButton;
```

```
  Button7: TButton;
```

```
  Button8: TButton;
```

```
  Edit1: TEdit;
```

```
  Edit2: TEdit;
```

```
  Edit3: TEdit;
```

```
  Image1: TImage;
```

```
  Image4: TImage;
```

```
  Label1: TLabel;
```

```
  Label3: TLabel;
```

```
  Label4: TLabel;
```

```

OpenPictureDialog1: TOpenPictureDialog;
SavePictureDialog1: TSavePictureDialog;
StringGrid1: TStringGrid;
t: TImage;
procedure Button1Click(Sender: TObject);
procedure Button2Click(Sender: TObject);
procedure Button5Click(Sender: TObject);
procedure Button6Click(Sender: TObject);
procedure Button7Click(Sender: TObject);
procedure FormClose(Sender: TObject);
private
  { private declarations }
public
  { public declarations }
end;

var
  Form1: TForm1;
implementation

{$R *.lfm}

{ TForm1 }
//процедура формирования ядра гауссовского ФНЧ
procedure gaussmask(var h:TA;kw,kh:integer;s:real;mustnormz:boolean);
//принимает массив для маски, его размеры, параметр рассеивания и
//нужно ли нормализовывать.
var i,j:integer;sum:real;
begin
  for i:=0 to kw-1 do
    for j:=0 to kh-1 do
      h[i,j]:=0;      //на всякий случай обнуляем
    if s<>0 then //если параметр рассеяния не равен 0
//вычисляем гауссоиду
      for i:=0 to kw-1 do
        for j:=0 to kh-1 do
          h[i,j]:=(1/(2*Pi*sqr(s)))*exp(-(sqr(i-kw div 2-1)+sqr(j-kh div 2-1))/(2*sqr(s)))

```

```
    else h[kw div 2,kh div 2]:=1;//если параметр рассеяния 0, то маски как бы нет.
```

```
    if mustnormz then //если нужно нормализовывать
```

```
    begin
```

```
        sum:=0;
```

```
        //вычисляем сумму всех элементов
```

```
        for i:=0 to kw-1 do
```

```
            for j:=0 to kh-1 do
```

```
                sum:=sum+h[i,j];
```

```
        //и каждый элемент делим на неё
```

```
        for i:=0 to kw-1 do
```

```
            for j:=0 to kh-1 do
```

```
                h[i,j]:=h[i,j]/sum;
```

```
        end;
```

```
    end;
```

```
//преобразование в формат pf24bit
```

```
function ConvertTo24Bit(aValue: TBitmap;BackColor: TColor): TBitmap;
```

```
begin
```

```
    Result := TBitmap.Create;
```

```
    with Result do
```

```
        if aValue.PixelFormat <> pf24bit then
```

```
        begin
```

```
            PixelFormat := pf24bit;
```

```
            SetSize(aValue.Width,aValue.Height);
```

```
            Canvas.FloodFill(0,0,BackColor,fsBorder);
```

```
            Canvas.Draw(0,0,aValue);
```

```
        end
```

```
    else
```

```
        Assign(AValue);
```

```
end;
```

```
//открытие файла, конвертация если необходимо
```

```
procedure TForm1.Button1Click(Sender: TObject);
```

```
begin
```

```
    if OpenPictureDialog1.Execute then
```

```
        t.Picture.LoadFromFile(OpenPictureDialog1.FileName);
```

```
        Label3.Caption:=IntToStr(t.Picture.Bitmap.Width);
```

```

Label4.Caption:=IntToStr(t.Picture.bitmap.Height);
if t.Picture.bitmap.PixelFormat <> pf24bit then
begin
    showmessage('invalid pixel color format. trying to convert');
    t.picture.bitmap.assign(convertto24bit(t.Picture.bit-
map,RGB(255,255,255)));
end;
end;

//выполнение задания 1.
procedure TForm1.Button2Click(Sender: TObject);
var i,j,k,l,kerheight,kerwidth,he,wi:integer;min,max,kp,s:real;rr,rg,rb,h:ta;
begin
kerwidth:=15;kerheight:=15;//размер ядра (не маленький)
he:=t.Picture.Bitmap.Height;//количество пикселей по вертикали
wi:=t.Picture.Bitmap.width;//количество пикселей по горизонтали
setlength(h,kerwidth,kerheight);//устанавливаем размер массива с ядром
свёртки
setlength(rr,wi,he);//устанавливаем размер массива с красным каналом
изображения
setlength(rg,wi,he);//устанавливаем размер массива с зелёным каналом
изображения
setlength(rb,wi,he);//устанавливаем размер массива с синим каналом
изображения
//во сколько раз увеличится параметр рассеяния к краю
kp:=6/sqrt(sqr(wi div 2-kerwidth div 2)+sqr(he div 2-kerheight div 2));
with t.Picture.Bitmap.Canvas do //с исходным изображением
begin
    for i:=0 to wi-1-kerwidth do //по всей ширине
    begin
        for j:=0 to he-1-kerheight do //по всей высоте
        begin
            rr[i,j]:=0;rg[i,j]:=0;rb[i,j]:=0;//на всякий случай
            //вычисляем параметр рассеяния в зависимости от расстояния
до центра
            s:=sqrt(sqr(i-wi div 2+kerwidth div 2)+sqr(j-he div 2+kerheight div
2))*kp;

```

```

        gaussmask(h,kerwidth,kerheight,s,true);//формируем массив с яд-
ром
        for k:=0 to kerwidth-1 do //осуществляется свёртка
            for l:=0 to kerheight-1 do
                begin
                    rr[i,j]:=rr[i,j]+getrvalue(Pixels[i+k,j+l])*h[k,l];
                    rg[i,j]:=rg[i,j]+getgvalue(Pixels[i+k,j+l])*h[k,l];
                    rb[i,j]:=rb[i,j]+getbvalue(Pixels[i+k,j+l])*h[k,l];
                end;
            end;
        //выводим индикатор, свёртка с таким размером ядра медленная
        button2.caption:='свёртка'+'inttostr(round(i/(width-ker-
width)*100))+'%';
    end;
end;
//на всякий случай вгоняем яркость в допустимый диапазон
min:=rr[0,0]; //для этого находим минимальное значение
for i:=0 to wi-1-kerwidth do
    for j:=0 to he-1-kerheight do
        begin
            if min>rr[i,j] then min:=rr[i,j];
            if min>rg[i,j] then min:=rg[i,j];
            if min>rb[i,j] then min:=rb[i,j];
        end;
    if min>=0 then min:=0;//если не выходим за пределы допустимого, из-
менений не будет
    max:=rr[0,0]-min;//находим максимальное значение
    for i:=0 to wi-1-kerwidth do
        for j:=0 to he-1-kerheight do
            begin
                if max<rr[i,j]-min then max:=rr[i,j]-min;
                if max<rg[i,j]-min then max:=rg[i,j]-min;
                if max<rb[i,j]-min then max:=rb[i,j]-min;
            end;
        //если не выходим за пределы допустимого, изменений не будет
        if max<=255 then max:=1 else max:=255/max;
        //формируем изображение с результатом
        with Image4.Picture.Bitmap do

```

```

begin
  Clear;
  PixelFormat:=pf24bit;
  Width:=wi;
  Height:=he;
  Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder);
  for i:=0 to wi-1-kerwidth do
    for j:=0 to he-1-kerheight do
      canvas.pixels[i+kerwidth div 2,j+kerheight div 2]:=RGB(round((rr[i,j]-min)*max),round((rg[i,j]-min)*max),round((rb[i,j]-min)*max));
    end
  end;

  //выборочное увеличение резкости
  procedure TForm1.Button5Click(Sender: TObject);
  var
    i,j,k,l,kerheight,kerwidth,he,wi:integer;minr,ming,minb,min,max,maxr,maxg,maxb,phi:real;ir,ig,ib,rr,rg,rb,h:ta;
  begin
    phi:=strtofloat(edit3.text);//пороговая величина контраста
    kerwidth:=3;kerheight:=3; //размер ядра свёртки
    he:=t.Picture.Bitmap.Height;//размер изображения
    wi:=t.Picture.Bitmap.width;
    setlength(h,kerwidth,kerheight);//массив для ядра
    setlength(rr,wi,he); setlength(rg,wi,he); setlength(rb,wi,he); //массивы с результатом
    setlength(ir,wi,he); setlength(ig,wi,he); setlength(ib,wi,he);//исходные данные
    h[0,0]:=-1; h[1,0]:=-1; h[2,0]:=-1; //ядро свёртки
    h[0,1]:=-1; h[1,1]:=9; h[2,1]:=-1;
    h[0,2]:=-1; h[1,2]:=-1; h[2,2]:=-1;
    with t.Picture.Bitmap.Canvas do //с исходным изображением
      begin
        for i:=0 to wi-1 do //заполняем массивы с исходными данными и результатом
          for j:=0 to he-1 do
            begin

```

```

    ir[i,j]:=getrvalue(Pixels[i,j]);
    ig[i,j]:=getgvalue(Pixels[i,j]);
    ib[i,j]:=getbvalue(Pixels[i,j]);
    rr[i,j]:=ir[i,j];rg[i,j]:=ig[i,j];rb[i,j]:=ib[i,j];
end;
for i:=0 to wi-1-kerwidth do //рассматриваем каждый элемент ре-
зультата
begin
    for j:=0 to he-1-kerheight do
    begin
        minr:=ir[i,j];ming:=ig[i,j];minb:=ib[i,j];//вычисляем минимумы
        maxr:=ir[i,j];maxg:=ig[i,j];maxb:=ib[i,j];//и максимумы для
        for k:=0 to kerwidth-1 do //вычисления контраста
            for l:=0 to kerheight-1 do
            begin
                if minr > ir[i+k,j+l] then minr:=ir[i+k,j+l];
                if ming > ig[i+k,j+l] then ming:=ig[i+k,j+l];
                if minb > ib[i+k,j+l] then minb:=ib[i+k,j+l];
                if maxr < ir[i+k,j+l] then maxr:=ir[i+k,j+l];
                if maxg < ig[i+k,j+l] then maxg:=ig[i+k,j+l];
                if maxb < ib[i+k,j+l] then maxb:=ib[i+k,j+l];
            end;
            for k:=0 to kerwidth-1 do //осуществляем свёртку
                for l:=0 to kerheight-1 do //если контраст больше порога
                begin
                    if (maxr-minr)>phi then rr[i,j]:=rr[i,j]+ir[i+k,j+l]*h[k,l];
                    if (maxg-ming)>phi then rg[i,j]:=rg[i,j]+ig[i+k,j+l]*h[k,l];
                    if (maxb-minb)>phi then rb[i,j]:=rb[i,j]+ib[i+k,j+l]*h[k,l];
                end;
            end;
        end;
    end;
end;
//если яркость выходит за пределы допустимого, вычисляем пара-
метры нормировки.
min:=rr[0,0];
for i:=0 to wi-1-kerwidth do
    for j:=0 to he-1-kerheight do
        begin

```

```

        if min>rr[i,j] then min:=rr[i,j];
        if min>rg[i,j] then min:=rg[i,j];
        if min>rb[i,j] then min:=rb[i,j];
    end;
    if min>=0 then min:=0;
    max:=rr[0,0]-min;
    for i:=0 to wi-1-kerwidth do
        for j:=0 to he-1-kerheight do
            begin
                if max<rr[i,j]-min then max:=rr[i,j]-min;
                if max<rg[i,j]-min then max:=rg[i,j]-min;
                if max<rb[i,j]-min then max:=rb[i,j]-min;
            end;
        if max<=255 then max:=1 else max:=255/max;
        with Image1.Picture.Bitmap do //выводим результат в Image1
            begin
                Clear;
                PixelFormat:=pf24bit;
                Width:=wi;
                Height:=he;
                Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder);
                for i:=0 to wi-1-kerwidth do
                    for j:=0 to he-1-kerheight do
                        canvas.pixels[i+kerwidth div 2,j+kerheight div
2]:=RGB(round((rr[i,j]-min)*max),round((rg[i,j]-
min)*max),round((rb[i,j]-min)*max));
                    end;
                end;
            //сохраняем размытое изображение
            procedure TForm1.Button6Click(Sender: TObject);
            begin
                if SavePictureDialog1.Execute then
                    Image4.Picture.SaveToFile(SavePictureDialog1.FileName);
                end;
            //сохраняем изображение повышенной чёткости
            procedure TForm1.Button7Click(Sender: TObject);
            begin
                if SavePictureDialog1.Execute then

```

```
Image1.Picture.SaveToFile(SavePictureDialog1.FileName);  
end;
```

```
procedure TForm1.FormClose(Sender: TObject);  
begin  
  //освобождаем память  
  t.Free;  
  image1.free;  
  image4.Free;  
end;
```

```
end.
```

Запускаем программу и получаем такой вот результат (рис.5):



Рис. 5. Форма для программы фильтрации изображений